

BCSE204L Design and Analysis of Algorithms

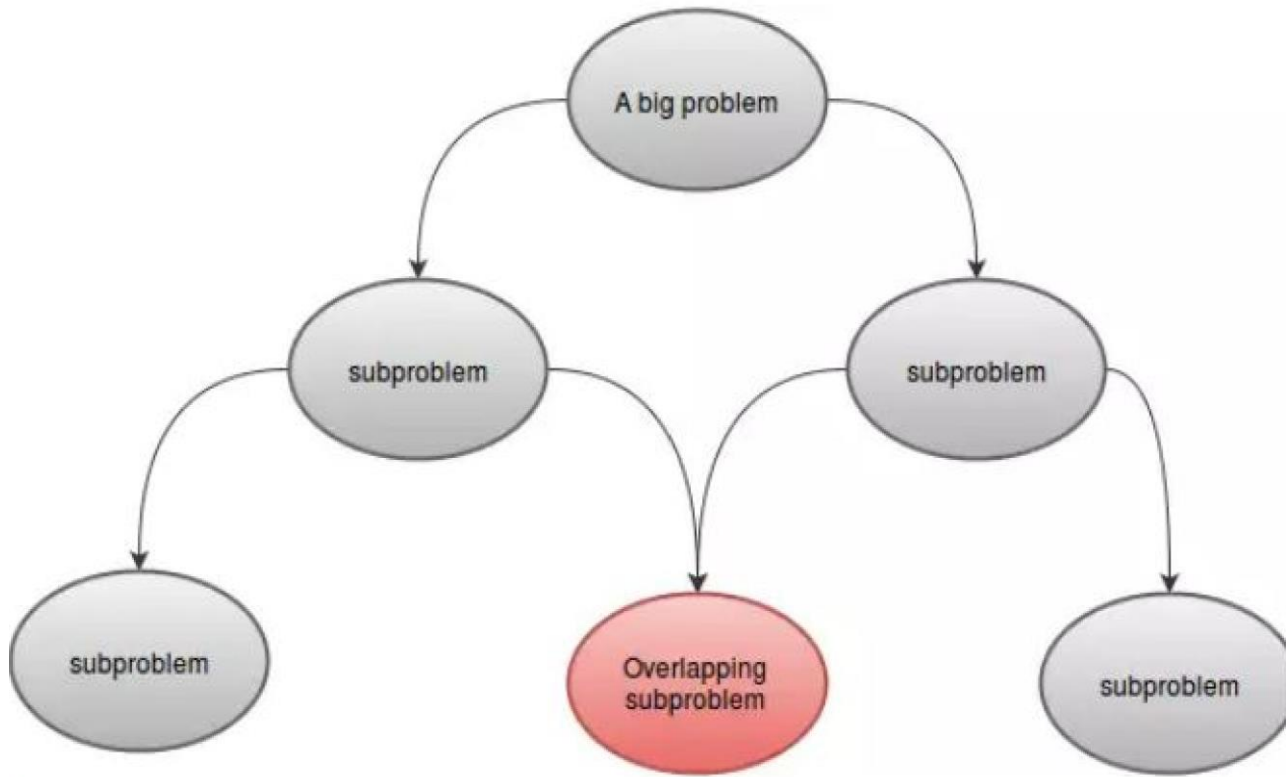
Module-2

Topic-1: Dynamic Programming

Dynamic Programming (DP) Basics

- Programming here is **tabulation**, not coding
- For solving optimization problems
- Like divide and conquer
- But applied when subproblems overlap
- That is, when subproblems share subsubproblems.

Typical DP Scenario



A problem can be solved using Dynamic Programming if it has some characteristics:

1.Subproblems

2.Overlapping Subproblems

3.Optimal Substructure

When to apply DP?

- If it is not possible to make a sequence of stepwise decisions leading to an optimal decision sequence, then try all possible decision sequence
- In such cases, enumerate all decision sequence and then pick out the best
- But the time and space requirements may be prohibitive.
- Dynamic programming often drastically reduces the amount of enumeration by avoiding the enumeration of some decision sequences that can not possibly be optimal

DP Features

- DP tries out all possible solutions and finds the optimal one
- DP uses recursive formula
- DP follows principle of optimality. ie., solve problem by taking sequence of decisions to get optimal solution (in every step)

Optimal Substructure in DP

- Key principle in DP
- A given problem is said to have **Optimal Substructure Property** if an optimal solution can be constructed from optimal solutions of its subproblems
- Also known as the **Principle of Optimality**

Example - 1

- The **shortest path problem** satisfies the Principle of Optimality.
- This is because if $a, x_1, x_2, \dots, x_n, b$ is a shortest path from node a to node b in a graph, then the portion of x_i to x_j on that path is a shortest path from x_i to x_j .

Example - 2

- The **longest path problem**, on the other hand, does not satisfy the Principle of Optimality.
- E.g.: The undirected graph G of nodes a, b, c, d , and e , and edges (a, b) (b, c) (c, d) (d, e) and (e, a) .
- G is a ring.
- The longest (noncyclic) path from a to d is a, b, c, d . The subpath from b to c on that path is simply the edge (b, c) . But that is not the longest path from b to c . Rather, b, a, e, d, c is the longest path.
- Thus, the subpath on a longest path is not necessarily a longest path.

Greedy and Principle of Optimality

- Typically, a greedy algorithm is used to solve a problem with optimal substructure if it can be proven by induction that this is optimal at each step.
- Otherwise, provided the problem exhibits overlapping subproblems, divide-and-conquer methods or dynamic programming may be used.

DP vs D&C

- Inherently, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subsubproblems, a disadvantage of D&C
- Dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem.

DP vs D&C [cont..]

- Why did recursion work so well with divide-and-conquer?
- A problem is expressed in terms of subproblems that are substantially smaller, say half the size.
- Because of this drop in problem size, the full recursion tree has only logarithmic depth and a polynomial number of nodes
- When does dynamic programming method enter the picture?
- A problem is reduced to subproblems that are only slightly smaller—say P_j relies on P_{j-1} .
- Because of this, recursion tree generally has polynomial depth and an exponential number of nodes.

DP vs D&C [cont..]

- Both techniques split their input into parts, find subsolutions to the parts, and synthesize larger solutions from smaller ones.
- Divide and Conquer splits its input at prespecified deterministic points (e.g., always in the middle)
- Dynamic Programming splits its input at every possible split points rather than at a pre-specified points. After trying all split points, it determines which split point is optimal.

DP vs Greedy

- Both used to solve optimization problems
- Greedy follows predefined procedure
- D.P. tries out all possible solutions and finds the optimal one
- D. P. uses recursive formula
- D. P. follows principle of optimality. ie., solve problem by taking sequence of decisions to get optimal solution (in every step)
- In greedy one-time decision making

DP vs Greedy [cont..]

- In the greedy method only one decision sequence is ever generated
- In dynamic programming, many decision sequences may be generated. However, sequences containing suboptimal subsequences can not be optimal (if the principle of optimality holds) and so will not (as far as possible) be generated
- In DP optimal solutions to subproblems are retained to avoid recomputing their values
 - Memoization and tabulation
 - A shortest path example

DP vs Greedy [cont..]

- Both techniques are optimization techniques, and both build solutions from a collection of choices of individual elements.
- The greedy method computes its solution by making its choices in a serial forward fashion, never looking back or revising previous choices.
- Dynamic programming computes its solution bottom up by synthesizing them from smaller subsolutions, and by trying many possibilities and choices before it arrives at the optimal set of choices.
- There is no a priori litmus test by which one can tell if the Greedy method will lead to an optimal solution.
- By contrast, there is a litmus test for Dynamic Programming, called The Principle of Optimality

Implementation

- DP can be done by
 - Memoization
 - Tabulation
- Both will use tables to store the results of subproblems
- The results from future invocations of the same subproblem can be directly fetched from the table
- A lot of function calls are avoided
- In memorization we preserve the recursive nature of the code
- Tabulation will be purely iterative

An Example: Fibonacci Sequence

RECFIBO(n):

if $n = 0$

 return 0

else if $n = 1$

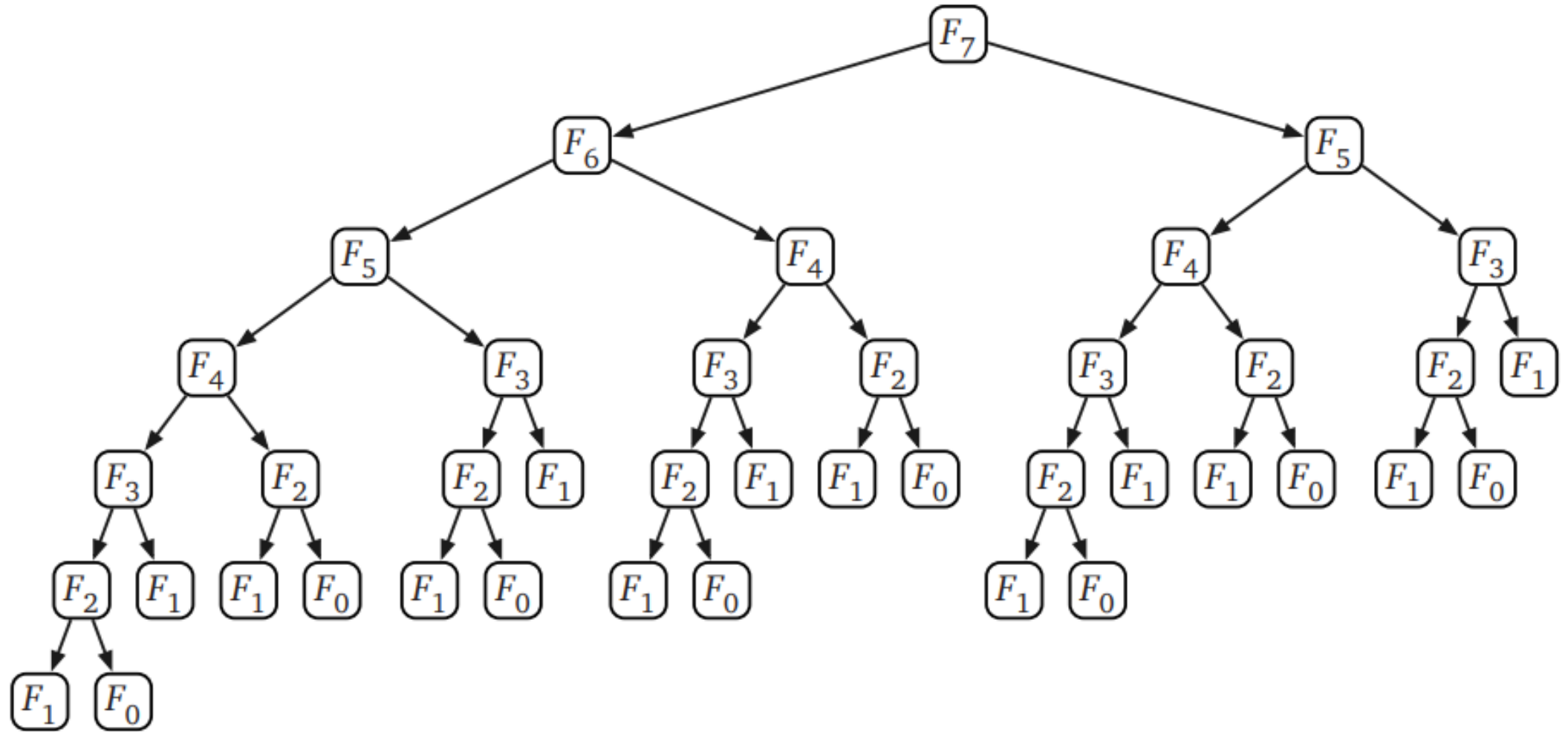
 return 1

else

 return RECFIBO($n - 1$) + RECFIBO($n - 2$)

- Recursive code
- The recurrence is $T(n) = T(n - 1) + T(n - 2)$
- Will take exponential complexity

Recurrence Tree



Memoization (Store the already computed results) – Recursive code

MEMFIBO(n):

if $n = 0$

return 0

else if $n = 1$

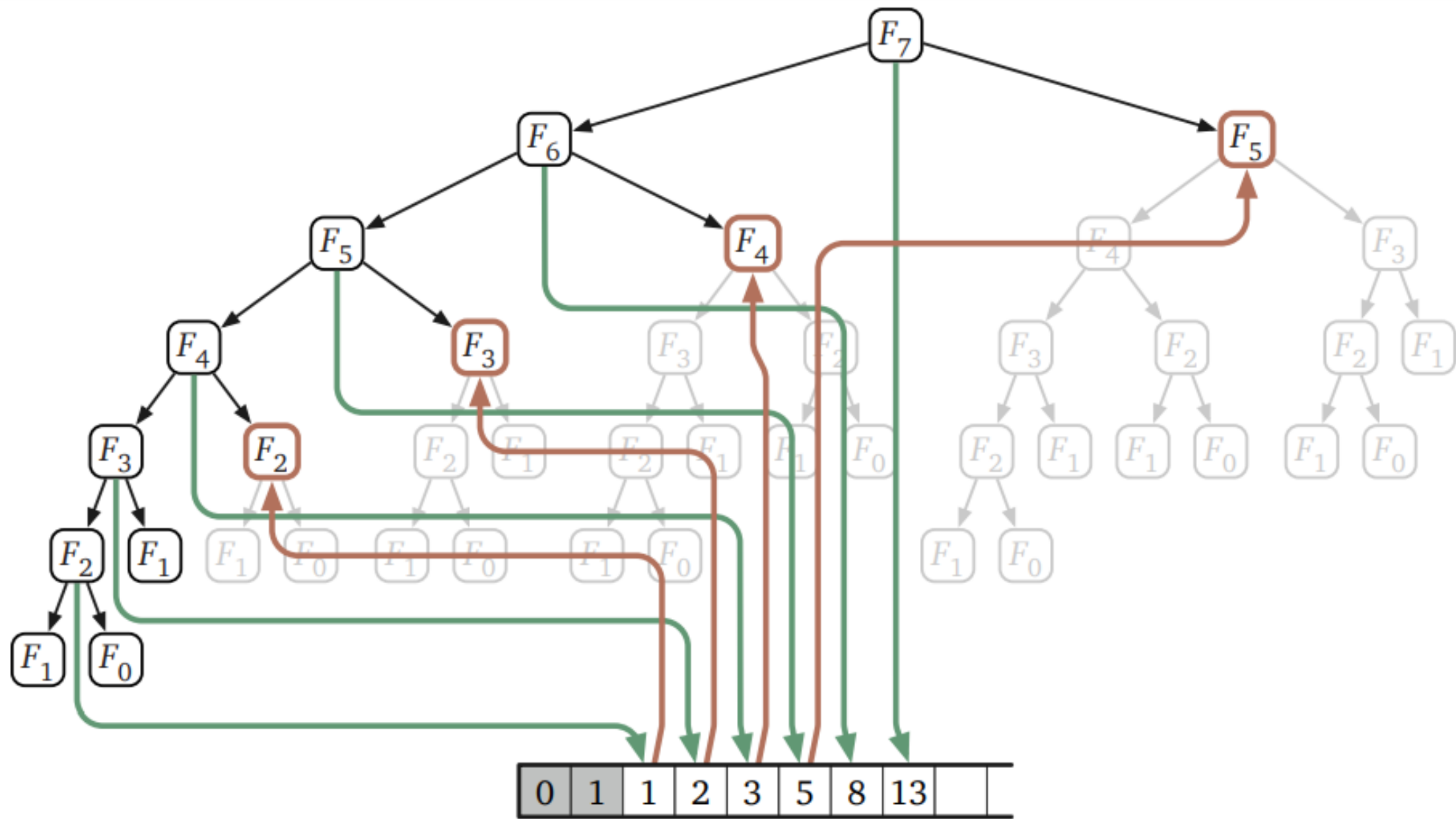
return 1

else

if $F[n]$ is undefined

$F[n] \leftarrow \text{MEMFIBO}(n - 1) + \text{MEMFIBO}(n - 2)$

return $F[n]$



Memoization Principles

- This code is still recursive
- It follows a top-down strategy
- Less common
- Start by solving the problem in a natural manner
- Store the solutions of the subproblems along the way
- Always hope that the solutions for the subproblem are already calculated
- If not, then we calculate them on the way

Tabulation Principles

- Follows bottom-up strategy
- Starting from the bottom i.e., start by calculating the 2nd term and then 3rd and so on and finally calculating the higher terms on the top by using these values
- The code will be iterative
- The most common way of solving problems using DP

Tabulation Pseudocode (iterative)

ITERFIBO(n):

$F[0] \leftarrow 0$

$F[1] \leftarrow 1$

for $i \leftarrow 2$ to n

$F[i] \leftarrow F[i - 1] + F[i - 2]$

return $F[n]$

Reduction in complexity

- Originally Fibonacci algorithm will take $O(2^n)$ complexity
- Whereas the DP versions (memoization and tabulation) will take linear complexity
- At the expense of additional space (arrays/DP tables)

DP Examples

- 0/1 knapsack
- Floyd-Warshall algorithm (All pair shortest path)
- LCS (Longest Common Subsequence)
- TSP (Travelling Salesman Problem)
- Matrix chain multiplication
- Assembly Line Scheduling
-and many more

References

- Algorithms by Jeff Erickson
(<https://jeffe.cs.illinois.edu/teaching/algorithms/>)
- <https://www2.seas.gwu.edu/~ayoussef/cs6212/dynamicprog.html>

Thank you..